📖 SRS Document Presentation Guide

AI Trip Planner Agent

This guide helps you understand and present your Software Requirements Specification (SRS) document.

🎯 What is an SRS Document?

An SRS (Software Requirements Specification) is a **blueprint for your software project** that answers:

- **WHAT** the system will do (not HOW to build it)
- **WHO** will use it
- **WHERE** it will run
- **WHEN** things should happen (timing, performance)
- **WHY** certain decisions were made

Think of it as a **contract between developers and stakeholders**.

📋 Your SRS Structure (Simple Explanation)

Section 1: Introduction (Page 1)

"Setting the stage"

What it contains:

- **Purpose:** Who reads this? (Developers, testers, managers)
- **Scope:** What does the app do? What does it NOT do?
- **Terms:** Definitions of technical words (AI, API, LLM)
- **References:** Links to related documents

Key Points to Highlight:

- ✅ App generates AI-powered trip itineraries
- ❌ App does NOT book hotels or flights
- 🎯 Goal: Reduce planning time from hours to minutes

Section 2: Overall Description (Pages 2-3)

"The big picture"

What it contains:

- **Product Perspective:** Is this new or part of something bigger? (It's **standalone**)
- **Functions:** Main features at high level
- **Users:** Who uses it? (Trip planners + developers)
- **Environment:** What devices/browsers does it work on?
- **Constraints:** Technology and business limitations
- **Dependencies:** What external services we rely on

Key Points to Highlight:

- 🌐 Web-based application (runs in browser)
- 🤖 Uses AI agents with LangChain framework
- ☁️ Needs internet and external APIs (Groq, OpenStreetMap, Weather)
- 📱 Works on mobile, tablet, and desktop

Section 3: Specific Requirements (Pages 4-10)

"The detailed specs"

This is the **CORE** of the SRS. It has 3 main parts:

3.1 Functional Requirements (FR-1 to FR-62)

"What the system MUST do"

Categories:

1. **Trip Planning** (FR-1 to FR-10)

- Accept destination, dates, budget
- Validate inputs (dates, budget)
- Generate day-by-day itinerary

2. **AI Agent** (FR-11 to FR-16)
   - Use LangChain ReAct pattern
   - Access 4 tools: coordinates, weather, POI, routes
   - Limit to 15 iterations
3. **Geocoding** (FR-17 to FR-20)
   - Convert "Paris" → GPS coordinates
4. **Weather** (FR-21 to FR-25)
   - Get forecast for all trip days
   - Adjust recommendations based on weather
5. **Points of Interest** (FR-26 to FR-30)
   - Find attractions, restaurants, museums
   - Max 50 results
6. **Routes** (FR-31 to FR-35)
   - Calculate distances between locations
7. **Recommendations** (FR-36 to FR-39)
   - Packing lists, weather tips, local insights
8. **Budget** (FR-40 to FR-44)
   - Estimate costs per activity
   - Track total spending
9. **API Endpoints** (FR-45 to FR-49)
   - `/api/plan-trip`, `/health`, `/docs`
10. **Error Handling** (FR-50 to FR-54)
    - Return proper error codes (400, 500, 503)
11. **Frontend** (FR-55 to FR-62)
    - Input form, loading spinner, display results

Example to Explain:

> **FR-1:** "The system shall accept trip planning requests via POST to `/api/plan-trip`"
> **Translation:** When a user fills the form and clicks "Plan Trip", the website sends data to the server using a POST request to a specific URL endpoint.

3.2 External Interface Requirements

"How the system connects with the outside world"

User Interface (UI):

- What the form looks like
- How results are displayed
- Loading indicators, error messages

Software Interfaces (APIs):

- **Groq API:** For AI brain (LLM)
- **OpenStreetMap:** For geocoding and POIs
- **Open-Meteo:** For weather data
- **OpenRouteService:** For routes (optional)

Communication:

- Uses HTTPS (secure)
- Data format: JSON
- CORS enabled for security

3.3 Non-Functional Requirements (NFR-1 to NFR-64)

"HOW WELL the system should work"

These define **quality**. Think of them as adjectives:

Category	What it Means	Examples
Performance	Speed	- Complete trip in 60 seconds - Handle 10+ users at once - Page loads in 1.5 seconds
Security	Safety	- API keys in environment variables - HTTPS in production - Rate limiting: 5 requests/min
Reliability	Trustworthy	- 99% uptime - Graceful failure handling

Category	What it Means	Examples
		- Retry failed API calls 3 times
Usability	Easy to use	- User-friendly errors - Real-time validation - Keyboard navigation
Maintainability	Easy to update	- Follow PEP 8 style - Modular code - 70% test coverage
Scalability	Handles growth	- Add more servers to handle traffic - Stateless design
Portability	Works everywhere	- Runs on Windows/Linux/Mac - Docker support - All modern browsers

Example to Explain:

> **NFR-1:** "System shall return itinerary within 60 seconds for 95% of requests"
> **Translation:** 95 out of 100 times, users get their trip plan in under a minute. This is **measurable** and **testable**.

3.4 Other Requirements

"Everything else important"

- **Legal:** Must credit OpenStreetMap
- **Compliance:** GDPR compliant (no data storage)
- **Data Management:** Cache results 24 hours, logs for 30 days
- **Documentation:** README, API docs, architecture guide
- **Deployment:** Setup in under 5 minutes
- **Monitoring:** Health checks, error logging

🎤 How to Present This SRS

For Non-Technical Audience (Managers, Stakeholders)

Focus on:

1. **Section 1.2 (Scope):** What the app does/doesn't do
2. **Section 2.2 (Functions):** Main features
3. **Section 3.1 (Functional):** Key capabilities (just mention the 11 categories)
4. **Section 3.3 (Non-Functional):** Performance, security highlights

Sample Script:

> "This SRS describes an AI Trip Planner that generates personalized itineraries in under 60 seconds. It integrates real-time weather and attractions using AI agents. The app doesn't book anything—it just plans. It's secure (HTTPS, rate limiting), fast (60-second responses), reliable (99% uptime), and works on any device."

For Technical Audience (Developers, Architects)

Focus on:

1. **Section 2.4 (Environment):** Tech stack details
2. **Section 2.5 (Constraints):** Technology decisions
3. **Section 2.6 (Dependencies):** External APIs and their SLAs
4. **Section 3.1 (Functional):** All 62 requirements (drill into specifics)
5. **Section 3.2 (Interfaces):** API endpoints, data formats
6. **Section 3.3 (Non-Functional):** Performance metrics, security measures

Sample Script:

> "The system uses FastAPI with LangChain agents running Llama 3.1 70B via Groq. It integrates 4 external APIs: Nominatim for geocoding, Open-Meteo for weather, Overpass for POIs, and OpenRouteService for routing with Haversine fallback. Frontend is vanilla JS, responsive 320px-4K. We have 62 functional requirements covering trip planning, AI orchestration, and UI. Non-functionals specify 60-second max response, 99% uptime, HTTPS in prod, rate limiting at 5 req/min, and WCAG 2.1 Level AA accessibility."

For QA/Testing Team

Focus on:

1. **Section 3.1 (Functional):** These are your test cases! Each FR = test scenario
2. **Section 3.3 (Non-Functional):** Performance benchmarks, security tests
3. **Section 3.2 (Interfaces):** API contract testing

Sample Script:

> "We have 62 functional requirements to test. For example, FR-2 through FR-6 define input validation—test boundary cases (dates, budget, string length). NFR-1 says 95% of requests must complete in 60 seconds—run load testing to verify. NFR-15 specifies rate limiting at 5 req/min—test with rapid requests to confirm blocking."

📊 Quick Metrics Summary

Requirements Breakdown:

- **62 Functional Requirements** (system behavior)
- **64 Non-Functional Requirements** (quality attributes)
- **5 User Interface specs**
- **6 Software Interface specs** (APIs)
- **3 Communication protocols**
- **Multiple** legal, compliance, and operational requirements

Total: ~140+ individual requirements to implement and test

✅ SRS Checklist (Is it Complete?)

Use this to verify your SRS:

- ☑ Defines purpose and scope clearly
- ☑ Lists all users and their needs
- ☑ Specifies operating environment
- ☑ Documents all constraints

- ☑ Lists all external dependencies
- ☑ Every functional requirement is:
  - ☑ Uniquely numbered
  - ☑ Testable (can verify it works)
  - ☑ Clear (no ambiguity)
- ☑ Non-functional requirements are measurable
- ☑ Defines all interfaces (UI, API, communication)
- ☑ Includes legal/compliance requirements
- ☑ Has glossary for technical terms
- ☑ References external documentation

Your SRS: ✅ Complete!

🔄 How to Use This SRS

During Development:

- Developers **implement** each FR
- Check off requirements as completed
- Use NFRs to guide code quality

During Testing:

- QA writes test cases from FRs
- Performance tests from NFRs
- API tests from interface specs

During Review:

- Stakeholders verify scope matches expectations
- Architects verify constraints are reasonable
- Legal verifies compliance requirements

For Maintenance:

- Update SRS when requirements change
- Version control (1.0, 1.1, 2.0, etc.)

- Use as reference for new developers

📚 Quick Reference: Key Sections

When You Need...	Go To Section...
Overview of what app does	1.2 Scope
List of features	2.2 Product Functions
Tech stack details	2.4 Operating Environment
External APIs used	2.6 Dependencies
Specific functionality	3.1 Functional Requirements
API endpoints	3.1.9 API Endpoints (FR-45 to FR-49)
UI design specs	3.2.1 User Interfaces
Performance targets	3.3.1 Performance
Security measures	3.3.2 Security
Data model	Appendix B

💡 Tips for Explaining SRS

1. **Start with the "Why":** Business problem → Solution → Requirements
2. **Use examples:** Turn technical requirements into real scenarios
3. **Show visuals:** Use the diagrams in your project (architecture, sequence)
4. **Be specific:** "60 seconds" not "fast", "99% uptime" not "reliable"
5. **Trace back:** Feature → Requirement → Test Case
6. **Keep it updated:** SRS is a living document

📞 Questions People Might Ask

Q: Why so many requirements?

A: Each requirement is testable and specific. Better to have 100 clear requirements than 10 vague ones.

Q: What's the difference between FR and NFR?

A: FR = *what* it does ("system shall send email"), NFR = *how well* it does it ("email within 60 seconds")

Q: How do you verify requirements?

A: FRs → functional tests, NFRs → performance/security tests, load testing, code reviews

Q: Can requirements change?

A: Yes! Update the SRS, increment version, track changes in Document Control table

Q: Why IEEE 830-1998 format?

A: Industry standard, comprehensive, familiar to most software teams

Good luck with your presentation! 🚀

For questions, refer to the full SRS document or contact the development team.